

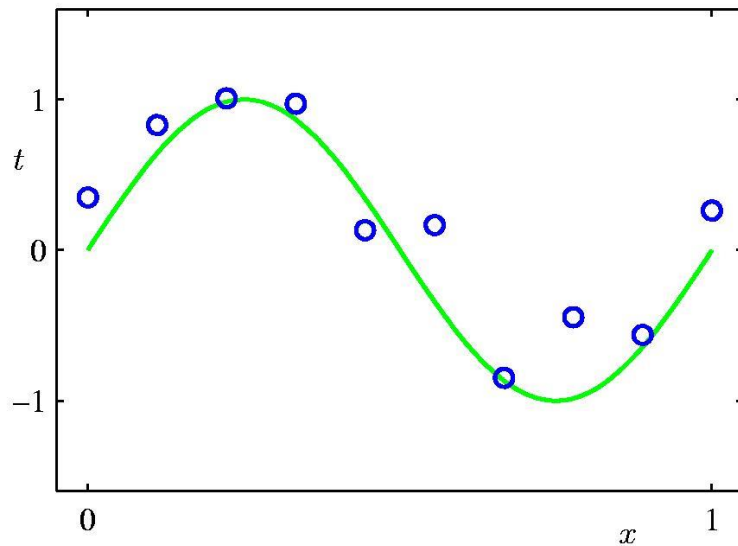
Machine Learning & Artificial Intelligence for Data Scientists: Regression (Part 3)

Ali Gooya

ali.gooya@glasgow.ac.uk

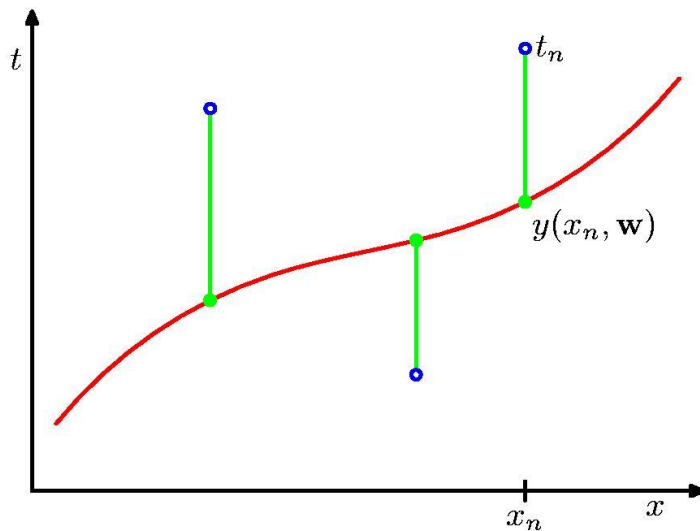
School of Computing Science

Polynomial Curve Fitting



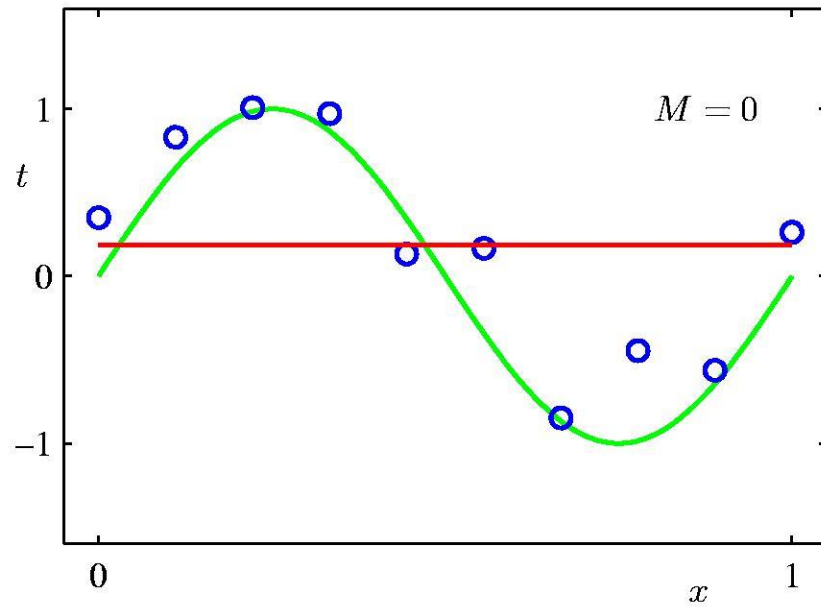
$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

Sum-of-Squares Error Function

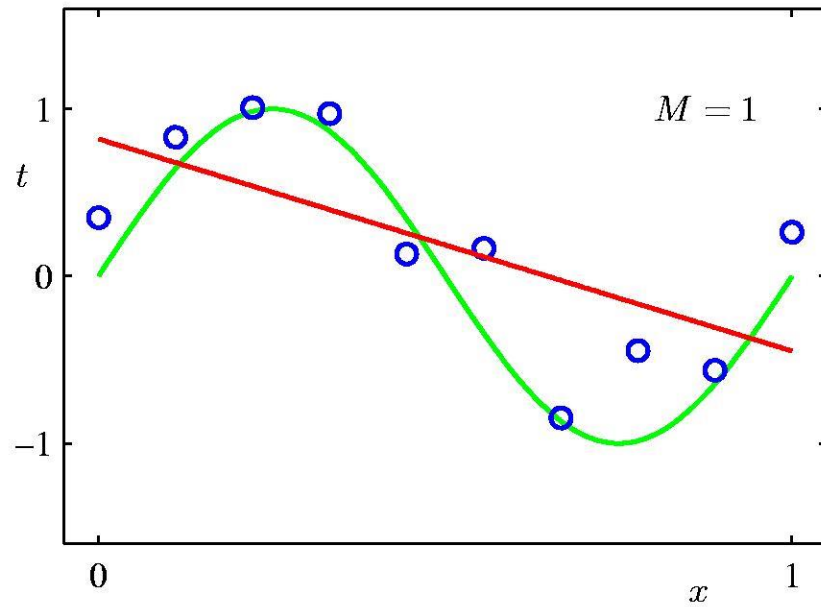


$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

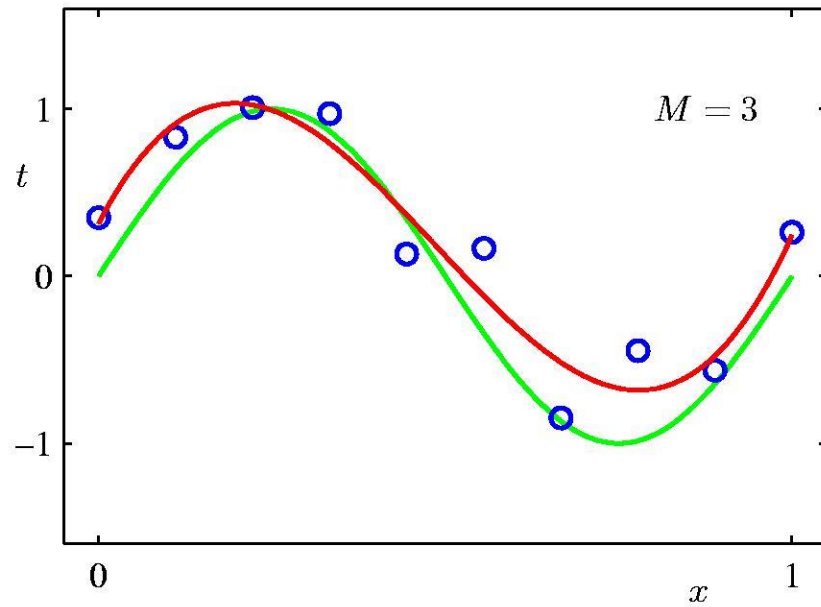
0th Order Polynomial



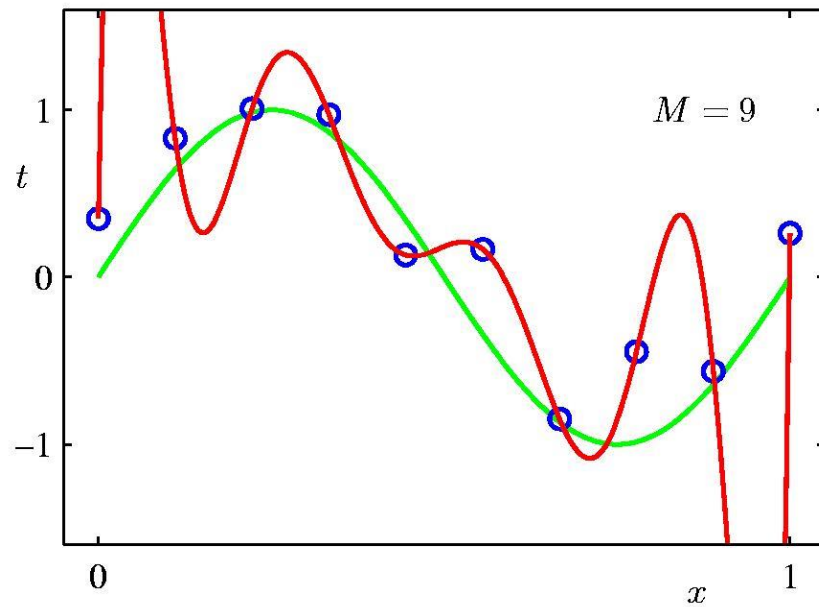
1st Order Polynomial



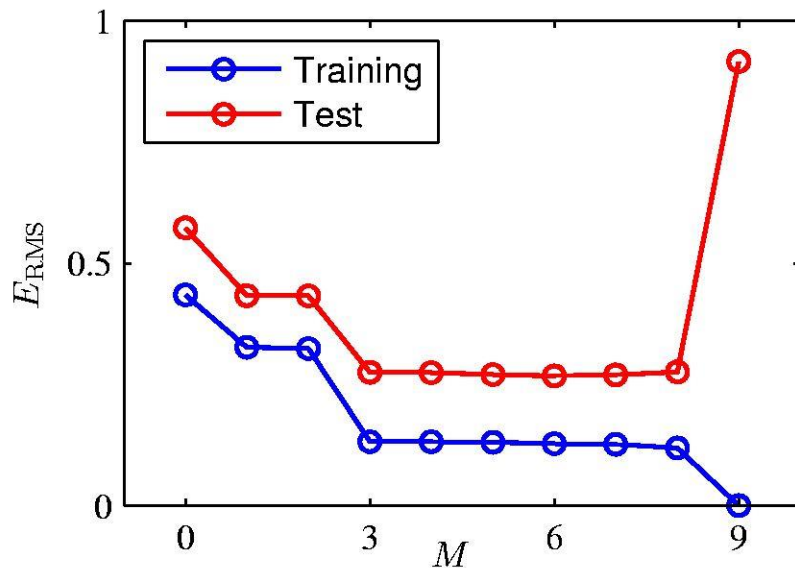
3rd Order Polynomial



9th Order Polynomial



Over-fitting



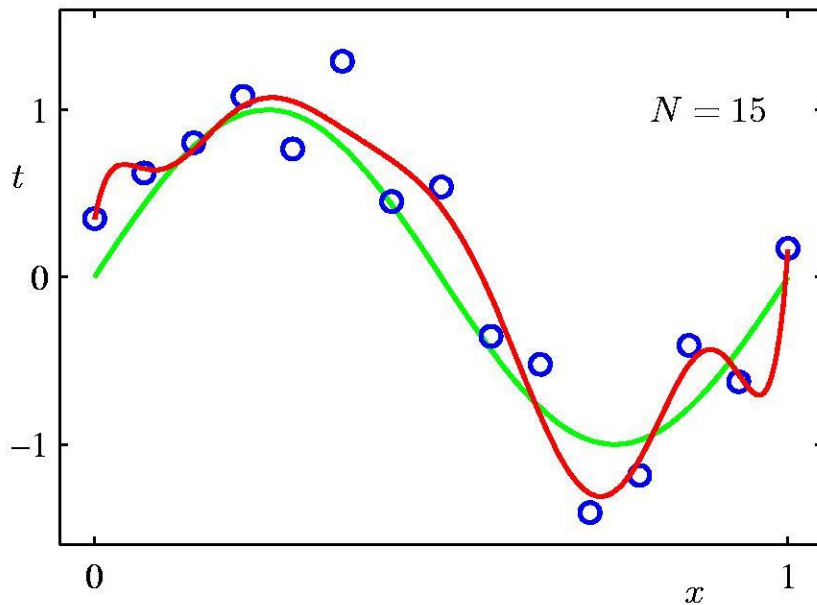
Root-Mean-Square (RMS) Error: $E_{\text{RMS}} = \sqrt{2E(\mathbf{w}^*)/N}$

Polynomial Coefficients

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

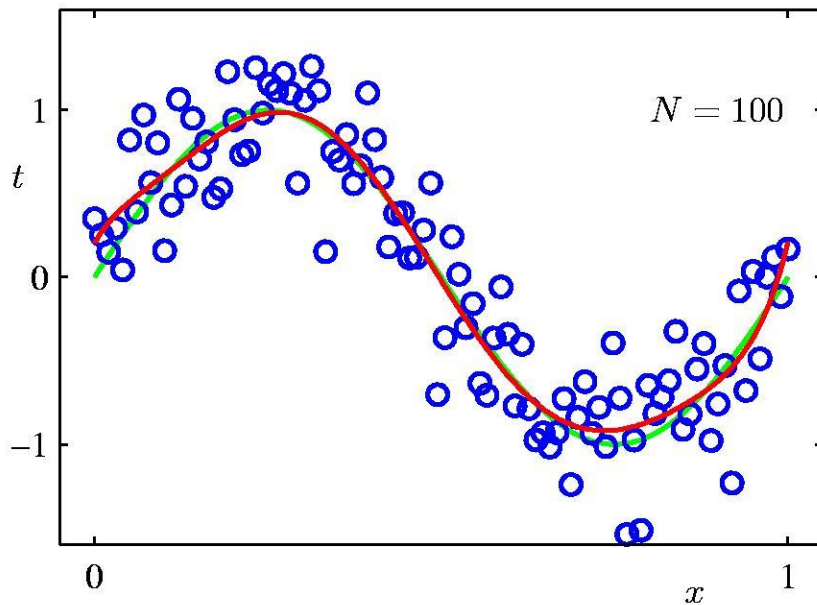
Data Set Size: $N = 15$

9th Order Polynomial



Data Set Size: $N = 100$

9th Order Polynomial



Regularized Least Squares (1)

- Consider the error function:

$$E_D(\mathbf{w}) + \lambda E_W(\mathbf{w})$$

Data term + Regularization term

- With the sum-of-squares error function and quadratic regularizer, we get

$$\frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$$

- which is minimized by

$$\mathbf{w} = \left(\lambda \mathbf{I} + \Phi^T \Phi \right)^{-1} \Phi^T \mathbf{t}.$$

λ is called the regularization coefficient.

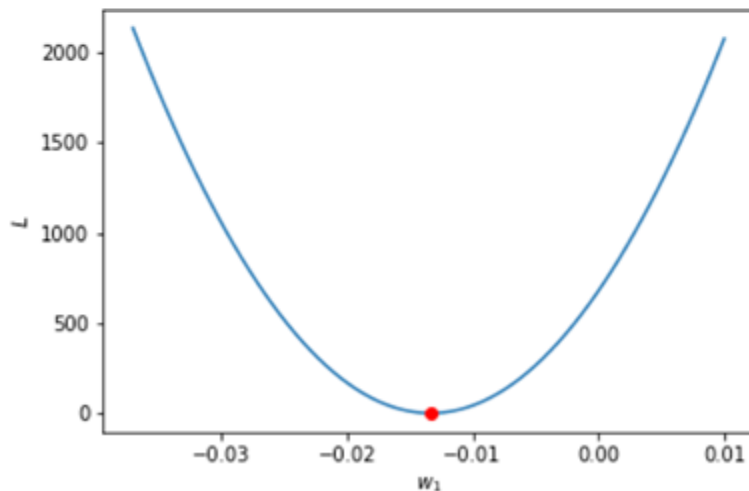
```

In [40]: L = np.zeros(num_candidates) # preallocating a vector for L e.g. np.zeros
for j in range(num_candidates): # For loop to evaluate L at every w1_candidates with w0 = 36.4164559025
    L[j] = np.mean( (t-36.4164559025-w1_candidates[j]*x)**2 ) # You can use the "np.mean" function

plt.plot(w1_candidates, L) # plot the resulting
plt.plot(-0.013330885711, 0.05, 'ro')
plt.xlabel('$w_1$')
plt.ylabel('$L$')

```

Out[40]: Text(0, 0.5, '\$L\$')



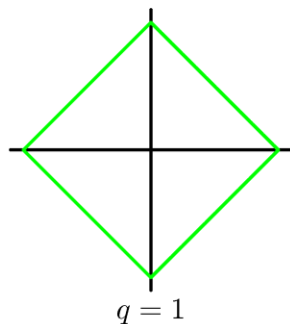
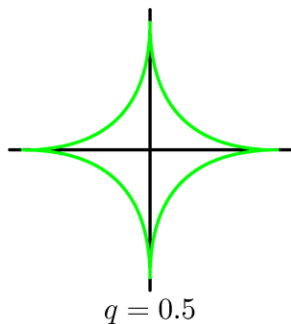
Let's look at the loss function in 1D

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{N} (\mathbf{t} - \mathbf{X}\mathbf{w})^T (\mathbf{t} - \mathbf{X}\mathbf{w})$$

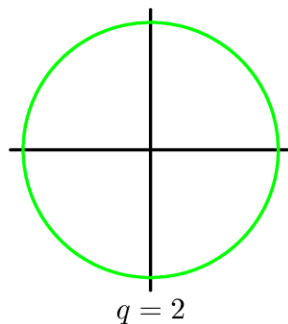
Regularized Least Squares (2)

With a more general regularizer, we have

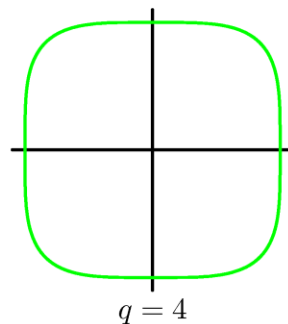
$$\frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \sum_{j=1}^M |w_j|^q$$



Lasso

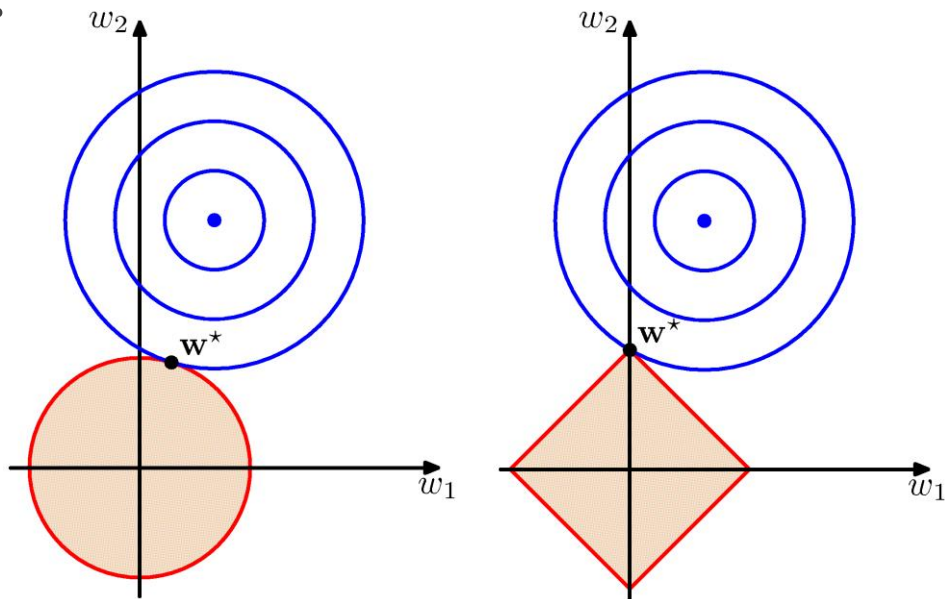


Quadratic/Ridge



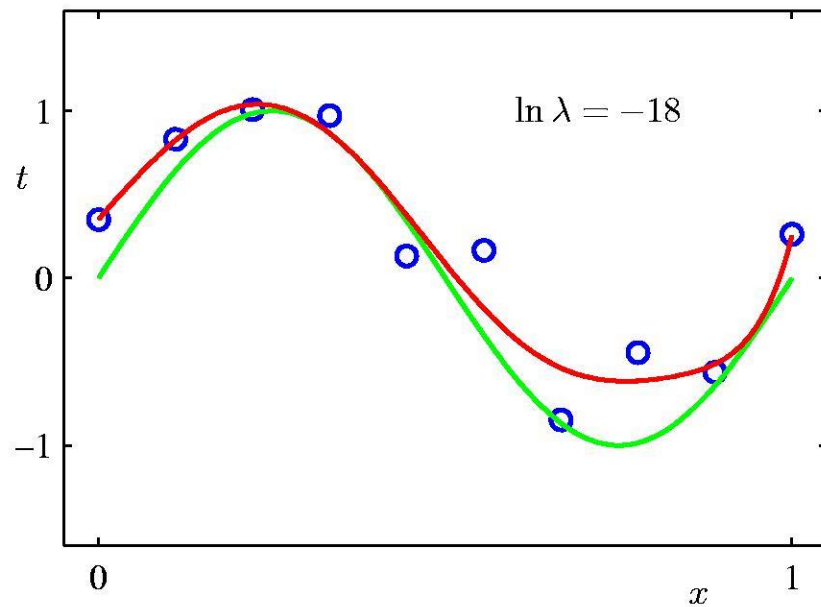
Regularized Least Squares (3)

Lasso tends to generate sparser solutions than a quadratic regularizer.



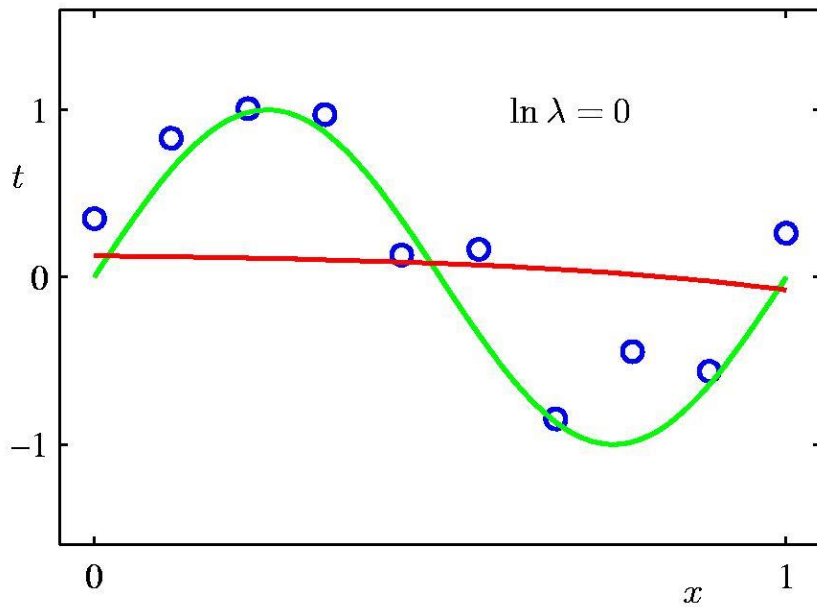
Regularization:

$$\ln \lambda = -18$$

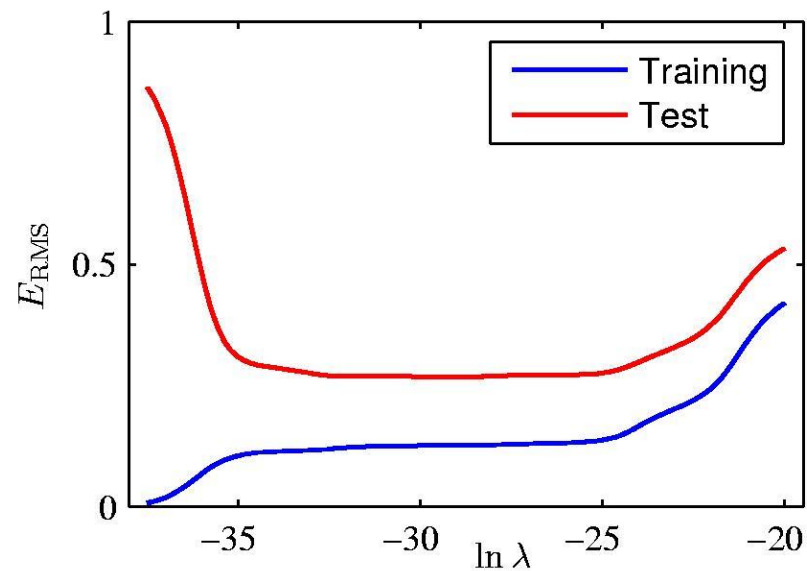


Regularization:

$$\ln \lambda = 0$$



Regularization: E_{RMS} VS. $\ln \lambda$



Polynomial Coefficients

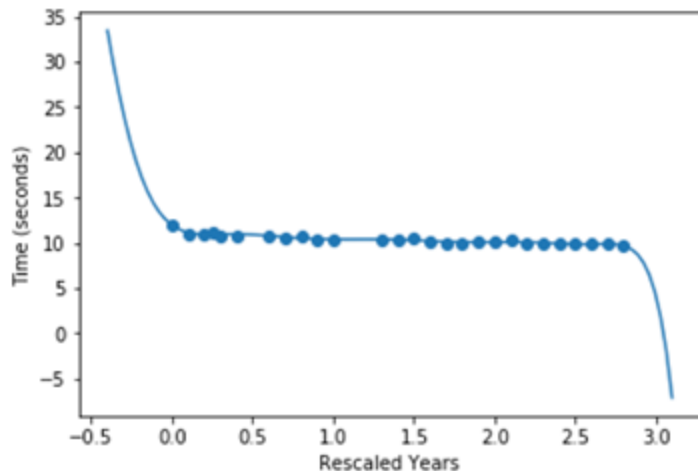
	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

```
In [87]: poly_order = 10
X_train = make_polynomial(x, poly_order)
X_test = make_polynomial(x_test, poly_order)

reg = LinearRegression()
reg.fit(X_train, t)

plt.plot(x_test, reg.predict(X_test))
plt.scatter(x,t) # draw a scatter plot
plt.xlabel('Rescaled Years') # always label x&y-axis
plt.ylabel('Time (seconds)') # always label x&y-axis
```

```
Out[87]: Text(0, 0.5, 'Time (seconds)')
```



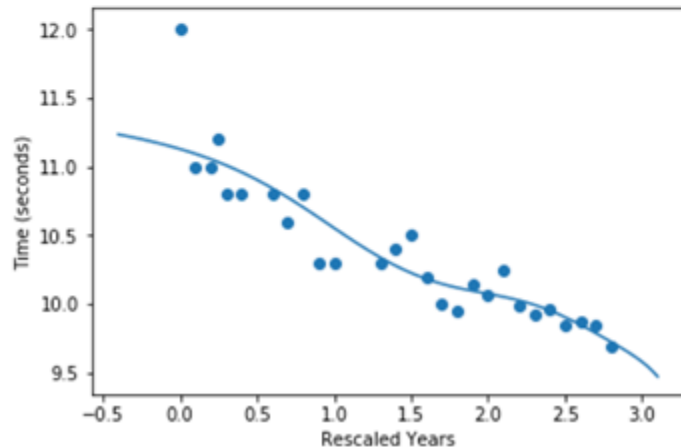
**Fit a polynomial
regression model
of order 10**

```
In [88]: from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

ridge = Ridge()
parameters = {'alpha': np.linspace(1, 10, 20)}
ridge_model = GridSearchCV(ridge, parameters, scoring = 'neg_mean_squared_error',
cv=5)
ridge_model.fit(X_train, t)

plt.plot(x_test, ridge_model.predict(X_test))
plt.scatter(x,t) # draw a scatter plot
plt.xlabel('Rescaled Years') # always label x&y-axis
plt.ylabel('Time (seconds)') # always label x&y-axis
```

```
Out[88]: Text(0, 0.5, 'Time (seconds)')
```



**Fit a ridge regression
model with α
determined by 5-fold CV**

Compare parameters between linear and ridge regression

— — —

```
In [89]: param_lr = reg.coef_[0]
param_ridge = ridge_model.best_estimator_.coef_[0]
param_lr[0] = reg.intercept_[0]
param_ridge[0] = ridge_model.best_estimator_.intercept_[0]
print(np.hstack((param_lr[:,None], param_ridge[:,None])))
```

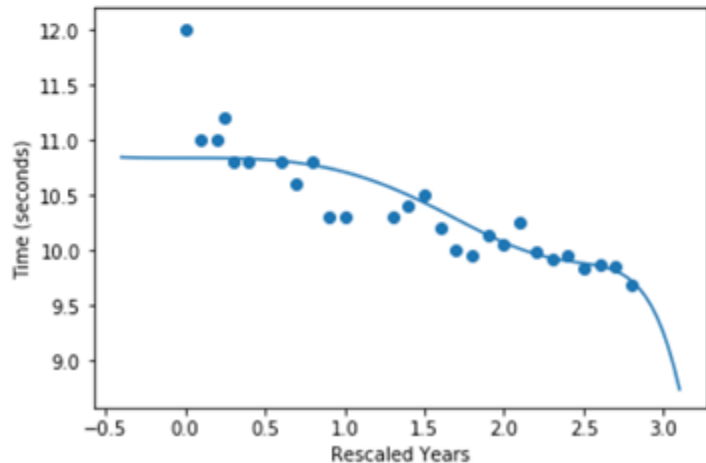
```
[[ 1.19648635e+01  1.11285803e+01]
 [-1.29443055e+01 -3.29359603e-01]
 [ 5.79522897e+01 -1.94725736e-01]
 [-1.09582035e+02 -9.64898104e-02]
 [ 6.23248849e+01 -1.87327081e-02]
 [ 7.48519704e+01  3.32402164e-02]
 [-1.46955431e+02  4.50182751e-02]
 [ 1.04735797e+02  9.53751777e-03]
 [-3.88035781e+01 -3.60588365e-02]
 [ 7.43343695e+00  1.40369595e-02]
 [-5.82870289e-01 -1.62830483e-03]]
```

```
In [90]: from sklearn.linear_model import Lasso

lasso = Lasso()
parameters = {'alpha': np.linspace(1e-1, 1, 20)}
lasso_model = GridSearchCV(lasso, parameters, scoring = 'neg_mean_squared_error',
cv=5)
lasso_model.fit(X_train, t)

plt.plot(x_test, lasso_model.predict(X_test))
plt.scatter(x,t) # draw a scatter plot
plt.xlabel('Rescaled Years') # always label x&y-axis
plt.ylabel('Time (seconds)') # always label x&y-axis
```

```
Out[90]: Text(0, 0.5, 'Time (seconds)')
```



**Fit a Lasso model with
 α determined by 5-
fold CV**

Compare parameters between linear regression, ridge regression, Lasso

— — —

```
In [91]: param_lasso = lasso_model.best_estimator_.coef_
param_lasso[0] = lasso_model.best_estimator_.intercept_[0]
print(np.hstack((param_lr[:,None], param_ridge[:, None], param_lasso[:,None])))
```

```
[[ 1.19648635e+01  1.11285803e+01  1.08381535e+01]
 [-1.29443055e+01 -3.29359603e-01 -0.00000000e+00]
 [ 5.79522897e+01 -1.94725736e-01 -0.00000000e+00]
 [-1.09582035e+02 -9.64898104e-02 -1.16126582e-01]
 [ 6.23248849e+01 -1.87327081e-02 -1.59001968e-02]
 [ 7.48519704e+01  3.32402164e-02 -0.00000000e+00]
 [-1.46955431e+02  4.50182751e-02  1.38119952e-03]
 [ 1.04735797e+02  9.53751777e-03  3.22128802e-03]
 [-3.88035781e+01 -3.60588365e-02  1.61616847e-04]
 [ 7.43343695e+00  1.40369595e-02 -8.65262203e-05]
 [-5.82870289e-01 -1.62830483e-03 -7.74413350e-05]]
```

What about a different loss?

$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} \sum_n^N |t_n - \mathbf{w}^T \mathbf{x}_n|^q$$

